

Frame It as Cases — Work That Speaks for Itself

Aditya Dixit | FlyRank AI Internship — General AI Fluency, Week 2 (Foundations)

VOICE CARD: direct, warm, plain, specific, no buzzwords

Added as standing instruction in Claude Project: "Write in Aditya's voice — direct, warm, plain, specific, no buzzwords. Use my words, keep sentences short, cut anything I wouldn't say out loud. No 'leverage,' 'robust,' or 'results-driven' filler."

How this was made

I didn't ask AI to "describe my work." I asked it to interview me — one question at a time — until the problem, my decisions, and what actually happened were out of me. I answered messy, then shaped those answers into three beats per case: **the problem** → **what I did (and decided)** → **what came of it**. Then I read every line out loud and cut anything I couldn't stand behind.

If a line could describe anyone's project, I rewrote it until it could only be mine.

CASE 1 — FlyRank Capstone: Settings Form (AI-assisted workflow drill)

- Context:** Frontend AI Engineering track, Week 2 — part of the FlyRank internship. Built in ``src/components/SettingsForm.jsx`` with React + Vite.
- The problem:** A "create a settings form with validation" prompt sounds simple, but the real work is what happens when validation is vague, accessibility is missing, and there are no tests to catch regressions. I needed a form that actually blocks bad data, tells screen readers what went wrong, and doesn't flip from controlled to uncontrolled when a user has only a partial profile.
- What I did — and what I decided:**
 - Picked the settings form as my small, capstone-relevant slice because it's where most portfolios hand-wave.
 - Round one was honest-lazy: one sentence, no file refs, no constraints. The AI gave me an imperative ``validateSettings()`` with regex and a ``useState`` bag of fields (``fullName/email/company/website/timezone``). No zod, no tests, generic error IDs.
 - Round two I forced a spec: file refs to ``src/utils/validateSettings.js`` and ``src/components/SettingsForm.jsx``, stack locked to ``react-hook-form + zod`` via `@hookform/resolvers/zod``, Tailwind, exact ``settings-name/email/notifications`` IDs, ``aria-invalid/aria-describedby`` + ``role=alert``, ``aria-busy``, ``empty-state/loading-state``, ``noValidate``, disabled states for ``isLoading/isSubmitting``, and the verification step: "write it, then write tests and run them" with a fresh session/branch.
 - Decided validation is a data contract, not UI logic — so I extracted ``settingsSchema`` (z.object), ``defaultSettings``, and ``hasSavedSettings()`` into ``src/utils/validateSettings.js`` and merged defaults safely: ``defaultValues: { ...defaultSettings, ...initialValues }``.
 - Had the model write 11 tests before I accepted the code, including the partial-values case that caught my first precise draft's bug.
- What came of it:**
 - Round one took ~2 min to generate but 35 min to review — I had to manually discover there were zero automated checks and that empty submit didn't consistently block.
 - Round two took ~11 min to spec + generate, 12 min to review because ``vitest run`` failed immediately on the partial-values bug before manual QA. End-to-end, the "slower" round was faster (23 min vs 37 min) and shippable. The form now blocks bad name/email with exact messages ("Name is required" / "Name must be at least 2 characters" / "Please enter a valid email address"), announces errors, disables during loading/saving with label "Saving...", and keeps inputs controlled even with ``{ name: 'Jane' }``.

- Pushed as `round-one-vague` vs `round-two-precise`; `WORKFLOW.md` (419 words) diffs them line by line, and `CLAUDE.md` now has 5 testable rules so the next form can't regress.

- inks: <https://github.com/Aditya-dxt/flyrank-capstone-aditya/tree/round-two-precise> • Live on Vercel (capstone build) — see `src/components/SettingsForm.test.jsx` for verification.*

CASE 2 — SneakerVault: Full-Stack E-Commerce Platform

- Stack:** React · Next.js · Node.js · MongoDB · Stripe · JWT | Jan–Apr 2026

- The problem:** Small sneaker sellers were juggling Instagram DMs, COD notebooks, and no real order history. I wanted one place where a buyer could browse, pay, and track — and a seller could manage inventory without calling a developer.

- What I did — and what I decided:**

- Chose Next.js for SEO-friendly product pages and a Node/Mongo MVC backend I could reason about, not the trendiest stack.

- Decided auth should be JWT with RBAC — not session cookies — so the admin dashboard routes could be protected cleanly on both client and API, and so I could debug token expiry without hidden server state.

- Built Stripe checkout end-to-end (cart → payment intent → webhook → order status) instead of a fake "pay" button, because a portfolio without a real transaction is a screenshot, not a product.

- Optimized Mongo queries (indexed lookups, lean returns) after noticing product list took >400ms — and decided to fix the production CORS bug on Vercel/Render myself rather than hide it, documenting the exact header fix.

- What came of it:**

- A working store with user/admin roles, secure sessions, and a complete order lifecycle from cart to status updates.

- API latency down ~35% on product reads after the query work.

- I can now explain every hard choice in that codebase — why RBAC lives in middleware, why webhooks update order state, why CORS broke and how I fixed it — which is the point of a case, not just a demo link.

CASE 3 — CivicSentinel: AI Civic Intelligence Portal

- Stack:** React · Node.js · Python · OpenAI API · RAG · LangChain · Geolocation API | Feb–Mar 2026 | Team project

- The problem:** Civic complaints (potholes, garbage, water issues) arrived as messy text, photos, and vague locations. Officials had no way to see patterns — just a pile of tickets.

- What I did — and what I decided:**

- Decided against a generic chatbot. We built a RAG pipeline: complaints are ingested, semantically classified, and embedded so a civic knowledge base can be queried with context, not keywords.

- Built "CivicCopilot" — semantic query over that base with context-aware LLM responses, so an officer can ask "where have water complaints clustered this week?" and get an answer grounded in real reports.

- Chose to use Geolocation API + reverse geocoding for auto-tagging incidents, and a dynamic knowledge graph to visualize city-issue relationships, because a table of tickets doesn't help a commissioner decide where to send a crew.

- Pushed for honest eval: we tracked where classification failed (informal Hindi/English mix) and tuned prompts rather than claiming "AI fixes governance."

- What came of it:**
 - Real-time ingestion → classification → insight loop that officials could test, not just read about.
 - National Finalist — India Innovates 2026 (national-level competition), which gave external validation beyond our own claims.
 - Learned to frame AI work as decisions and trade-offs, not magic — the story I now tell here: what we automated, what we kept human, and what we'd do better with better location data.
-

Bio

I'm Aditya Dixit — third-year B.Tech CS at PSIT Kanpur, building full-stack web apps with React, Next.js, Node, and Mongo. I care about AI that actually ships: RAG pipelines, not demos. 200+ DSA problems on LeetCode, 4-star Java on HackerRank. I've built e-commerce with real Stripe payments, civic AI that made a national final, and frontend that passes screen-reader tests before it ships. I write plainly and ship iteratively.

Contact / CTA

Want to see how I think through a problem, not just the final screen?

→ Email me: adityadxt1910@gmail.com — tell me one confusing flow in your product and I'll sketch how I'd simplify it. Or browse code: github.com/Aditya-dxt and portfolio at aditya-dixit.vercel.app.

Before / After — One Line, Same Idea

This is the difference between borrowing a generic story and using my own words.

- Before (generic AI line I asked it to generate):**

"As a passionate, results-driven developer leveraging cutting-edge technologies, I architected a robust, scalable, and innovative solution that delivers exceptional user experiences and drives transformative engagement."

I wouldn't say that out loud. It could be anyone. It proves nothing. "Leveraging," "robust," "transformative" — filler that hides the work.

- After (my edited version — what I actually say):**

"I built a settings form with react-hook-form + zod that blocks bad submits, tells screen readers exactly what's wrong, and passes 11 tests before I push — so the next person doesn't have to guess if it works."

Short, specific, testable. Only my form has `settings-name` IDs and that `hasSavedSettings` helper. That's the voice I'm keeping: direct, warm, plain, specific, no buzzwords.

- eliverable checklist: Voice card at top ✓ | One framed case per real piece (capstone + 2 shipped projects) with three beats ✓ | Bio + contact with one clear action ✓ | Before/after showing voice edit ✓ | No "results-driven" filler ✓*